

WUSP

Wantastic USP over WireGuard.

A control transport for USP semantics inside an authenticated WireGuard session.

The idea is simple. The tunnel already exists. It already authenticates peers. It already encrypts traffic. Do not build a second management plane around it. Build one inside it. WUSP carries USP-style device management over a dedicated WireGuard control class, using compact typed frames, bounded datagrams, request correlation, event delivery, and a TR-181 aligned data model. It is purpose-built for Wantastic. It is technically coherent. And it is not finished everywhere that production will eventually demand.

Authors	Abdelkarim Ouazmir <karim@wantastic.app> Adnane A. Zaid <adnane@wantastic.app>
Date	April 26, 2026
Status	Implementation-backed white paper based on the current Wantastic repository
Scope	Transport design, packet structure, controller mechanics, and production readiness

1. Design Principle

WUSP does not try to replace WireGuard. It assumes WireGuard has already solved the expensive problems: key agreement, authenticated peers, encrypted transport, endpoint roaming, and a small attack surface. WUSP begins after decryption. Its job is narrower and harder to fake: carry USP semantics cleanly over a tunnel that already belongs to the device.

That choice matters. Standard USP defines messages, records, endpoint identity, and management transport behavior [1]. WireGuard defines a simple authenticated VPN carried over UDP, with cryptokey routing and built-in roaming [2][3]. Wantastic uses both ideas, but not in their default composition. WUSP keeps the USP control vocabulary and TR-181 object model. It replaces the usual message transfer profile with a Wantastic transport that lives entirely inside a WireGuard session.

WUSP is not a new secure tunnel. It is a control plane inside one.

In a multi-vendor ecosystem, that would be a heavy trade. In a controlled WireGuard-native system, it is an efficient one. The control port disappears. Certificate exchange at the outer transport disappears. The management path inherits the same reachability as the overlay itself. The protocol surface gets smaller. The code path gets tighter.

2. Requirements

WUSP is shaped by seven requirements visible in the code.

1. Tunnel-only transport. No separate management listener.
2. USP-style operations: Get, Set, Add, Delete, GetInstances, Operate, Notify, capability discovery, transfer initiation.
3. TR-181 aligned state with Wantastic extensions under `Device.WUSP*`.
4. Bounded UDP-sized control traffic with explicit fragmentation when needed.
5. Typed binary payloads, not ad hoc JSON.
6. Version tolerance across controller and agent builds.
7. Operational behavior fit for dashboards: fast correlation, bounded waits, clear failure.

Those requirements are sensible. They also explain almost every implementation decision that follows.

3. Placement in the Stack

WUSP rides on a dedicated WireGuard message class. It is not sent as ordinary inner IP traffic. It is handled before IP routing on receive, and injected directly on send. This gives the device a clean split between overlay data and management control.

```
USP semantics
→ WUSP request / response transport
  → optional nested TR-181 message frame
    → optional WUSP control fragmentation
      → WireGuard message type 8
        → UDP
```

That split is one of the protocol's strongest choices. WireGuard already authenticates the peer before admitting traffic. WUSP then receives a payload that is inside a known secure session, tied to a WireGuard public key, with endpoint roaming already handled below it [2][3].

4. Packet Structure

4.1 WireGuard Control Class

The userspace WireGuard implementation assigns WUSP a dedicated transport class:

```
MessageWUSPType = 8
```

This is not cosmetic. It allows the receiver to distinguish device-management traffic from normal inner packets before IP dispatch. On the Wantastic side, that means WUSP can be processed as a native control plane, not as an application tunneled through a second protocol stack.

4.2 Control Fragment Envelope

Large control messages are split by WUSP itself rather than relying on IP fragmentation. That is the correct instinct. RFC 8085 is explicit that IP fragmentation lowers both efficiency and reliability, and that loss of one fragment loses the entire packet [4].

```
WUSP control fragment

byte 0      : magic      = 0x54
byte 1      : version    = 0x01
byte 2      : flags      = bit0 compressed
bytes 3-10   : message_id = uint64 LE
bytes 11-14  : index      = uint32 LE
bytes 15-18  : count      = uint32 LE
bytes 19-22  : raw_size   = uint32 LE
bytes 23..   : payload
```

The current implementation uses a conservative control datagram ceiling of 1200 bytes. Pre-fragment LZ4 compression is applied when it helps. That is a sensible engineering trade: fewer datagrams when the payload compresses well, no wasted CPU when it does not.

4.3 Control Transport Envelope

Inside the fragment payload is a compact transport envelope for USP operations. This is the real WUSP protocol surface.

WUSP transport header

```
byte 0      : version
byte 1      : kind          (1=request, 2=response)
byte 2      : method
byte 3      : flags
bytes 4-11  : request_id = uint64 LE
```

The envelope can carry path strings, compact path selectors, object selectors, metadata, nested TR-181 messages, transfer descriptors, supported data-model descriptions, and supported protocol descriptions. The method space covers:

```
Get | Set | Add | Delete | GetInstances | Operate | Notify
GetSupportedDM | GetSupportedProtocol | Upload | Download
```

4.4 Nested TR-181 Message Frame

WUSP uses its own typed message codec for parameter payloads. This is the most technically elegant part of the design. The frame is compact, schema-aware, and still version tolerant.

Nested message header (8 bytes)

```
bytes 0-1  : magic          = "WU"
byte 2     : version        = 0x01
bytes 3-4  : schema_fp     = FNV-16
byte 5     : flags          = compression, device_id, timestamp
bytes 6-7  : reserved
```

Fields are encoded with either a stable field ID or an inline path, followed by a one-byte type tag and a compact binary value. Supported value tags include booleans, integers, floats, strings, bytes, time, IPv4, IPv6, prefixes, MAC addresses, and lists.

4.5 Selector Compaction and String Fallback

The protocol also defines compact path selectors built from a stable numeric code plus concrete instance numbers. That is good design. But the code takes a pragmatic step further: nested parameter messages are emitted with string paths across the controller-agent boundary when necessary, because path-code registries can drift between independently built binaries. This is a very good example of protocol honesty. A smaller wire format is useless if it breaks on version skew.

5. Protocol Semantics

WUSP is USP-aligned, not USP-record compatible. It keeps the control semantics that matter and changes the transport underneath them.

TR-369 defines USP around CRUD-ON style messages and a formal object model [1]. WUSP preserves those semantics in a custom envelope:

- **Get** returns parameter values or object subtrees.
- **Set** applies field updates.

- **Add** creates instances of multi-instance objects.
- **Delete** removes parameters or object subtrees.
- **GetInstances** enumerates instantiated rows.
- **Operate** invokes command paths.
- **Notify** carries events and subscription management.
- **GetSupportedDM** and **GetSupportedProtocol** advertise capability.

Agent-originated events use the Notify path with stable metadata keys. This is how WUSP carries value changes, object creation and deletion, operation completion, and onboarding announcements. The event model is close enough to TR-369 to feel natural, while remaining independent of BBF protobuf records.

6. Data Model

The runtime model merges three surfaces: the imported TR-181 device model, the TR-181 WireGuard model, and Wantastic's own WUSP extension. This is not a small point. It means the protocol does not move opaque blobs; it moves device state with declared types, access modes, and object structure.

That choice is now even better justified than it would have been a few years ago. Broadband Forum's Device:2.20 model includes `Device.WireGuard.*` as a first-class object family [6]. Wantastic's WUSP extension sits next to that, not outside it. The result is a management plane that speaks in paths the controller already understands.

The most important Wantastic-specific parameters are the ones that describe the transport itself: enablement, status, protocol version, controller identity, compression support, recommended chunk size, transfer window, tunnel-only behavior, and reliability signaling.

7. Controller Mechanics

The paper would be incomplete without the controller, because that is where WUSP stops being a format and becomes a transport.

7.1 Request Correlation

Every outbound request carries a non-zero 64-bit ID. The controller keeps a pending map keyed by that ID until the response arrives or a timeout fires. This is simple. It is enough. It gives the dashboard deterministic request ownership without making the protocol stateful in the TCP sense.

7.2 Fragment Discipline

Incomplete fragment sets are buffered per peer, swept on a timer, and bounded in count. The current code uses a 30 second fragment TTL, a 10 second sweep interval, and a cap of 64 incomplete messages per peer. That is good operational hygiene. It prevents stale fragment buildup and makes memory costs predictable.

7.3 Reachability Heuristics

The controller tracks recent inbound WUSP activity and recent failures. After one response timeout or send failure, later operations can fast-fail instead of hanging the user through repeated full deadlines. This is not a standards feature. It is product intelligence. And it is the right thing to do.

7.4 Snapshot Strategy

One implementation comment deserves to be elevated into the paper: a full `Device.*` snapshot can explode into roughly hundreds of control fragments, while targeted state sync stays near a single-fragment response in normal cases. That is why the controller sync path deliberately fetches a curated subset of the tree. This is not a temporary shortcut. It is the protocol acknowledging the physics of UDP.

8. Security Model

WUSP inherits its transport security from WireGuard. That is a real advantage. WireGuard's public design emphasizes simple key configuration, authenticated transport packets, a small implementation footprint, and formal verification work on core security properties [2][3][7]. WUSP benefits from that foundation immediately.

But this is also where the paper needs to be precise. Standard USP spends substantial effort on certificate handling, endpoint IDs, trusted roles, and record security processing [1]. WUSP does not implement that exact transport trust story. Instead, it substitutes WireGuard peer identity as the outer trust anchor and models additional controller trust under `Device.WUSP.*`.

Today, that WUSP-specific trust model appears stronger in schema than in runtime enforcement. The transport is secure because WireGuard is secure and the overlay already authenticates peers. The finer-grained controller authorization model is not yet equally complete. That distinction matters.

9. Performance and Transport Discipline

UDP gives speed and simplicity. It also gives responsibility back to the application. RFC 8085 says so plainly: UDP is unreliable, best effort, and leaves message size, reliability, and congestion behavior to the application or encapsulating protocol [4]. RFC 8899 goes further and recommends Datagram PLPMTUD for robust size adaptation over datagram transports [5].

Against that standard, WUSP gets some things right today:

- it avoids raw IP fragmentation by fragmenting above WireGuard
- it keeps a conservative 1200 byte control budget
- it uses compression selectively
- it prefers targeted state fetches over giant full-tree pulls

And it leaves some things unfinished:

- no DPLPMTUD or equivalent active path-size adaptation
- no fragment-level ACK or replay for control messages
- no explicit circuit-breaker behavior beyond timeout-driven failure handling

Those are not academic omissions. They are exactly where controlled deployment and Internet-hard production begin to diverge.

10. Transfer Stream

WUSP defines a second frame family for larger uploads and downloads. It has its own magic value, version, session ID, request ID, sequence numbers, offsets, total size, metadata, and phase transitions:

```
Open → Chunk → Ack → Complete
                        ↘ Abort
```

This is the right architectural separation. Bulk transfer should not look like an oversized RPC frame. The stream format also supports selective LZ4 chunk compression when the data is likely to benefit.

But the honest evaluation is clear: in this repository, the transfer stream is less mature than the core control plane. The frame codec exists. Upload and download verbs exist. Transfer handlers exist on the agent abstraction. What is missing is strong evidence of a fully wired, controller-to-agent, stream-managed, portal-exposed, production-tested end-to-end path. Today the stream is a protocol design with partial implementation, not a finished transport subsystem.

11. Honest Evaluation

WUSP is not experimental in the casual sense. There is real engineering here. The control path is coherent. The data model is disciplined. The controller behavior is thoughtful. The protocol uses the WireGuard substrate intelligently instead of fighting it.

Its strongest decisions are these:

- put management inside the authenticated tunnel
- separate control traffic from inner IP payloads at the WireGuard layer
- use typed TR-181 frames instead of untyped blobs
- treat version skew as a design condition, not an accident
- optimize for real operator workflows, not protocol purity alone

Its weakest areas are also plain:

- large fragmented control responses remain loss-sensitive
- transport reliability is implied more than fully implemented
- bulk transfer is not yet at parity with the core control plane
- WUSP-specific trust enforcement is not as mature as the schema suggests
- the protocol is private and wire-incompatible with standard USP MTPs

The right conclusion is not “WUSP is unfinished.” The right conclusion is “WUSP is strong where it is narrow, and still incomplete where it becomes transport.”

12. Production Readiness for Wantastic

For Wantastic, the answer is not binary.

Yes, WUSP is credible for controlled production use today if the workload is targeted device management: narrow state sync, low-volume Get and Set, live event delivery, controller operations, and snapshot-oriented flows.

No, it is not yet ready to be described as fully production-hardened for arbitrary large control exchanges, broad Internet path variability, or a complete bulk-transfer story.

What should ship before broad production

1. **Path-size adaptation.** Implement RFC 8899 style DPLPMTUD or an equivalent probing strategy. Static ceilings are helpful. They are not enough.
2. **Fragment recovery.** Add control-fragment ACK, replay, or another deterministic recovery mechanism. Timeouts alone are not transport completeness.
3. **Circuit breaker behavior.** RFC 8085 recommends protective behavior for UDP applications and tunnels that do not implement full congestion control [4]. WUSP should add it explicitly.
4. **Truthful capability advertisement.** Do not advertise reliable transfer as a fact until the transfer path is fully deployed and tested end to end.
5. **Real controller authorization.** Make controller public key, endpoint identity, and trust-role logic part of runtime access decisions, or document clearly that WireGuard identity is the only authority.
6. **Compatibility testing.** Run older and newer controller-agent builds against each other for selector decoding, inline path fallback, and schema drift.
7. **Loss and soak testing.** Test with packet loss, duplication, reordering, roaming, and long-lived sessions. A UDP control plane must prove itself under stress.
8. **Observability.** Export fragment counts, timeout rates, response-size distributions, path-size downgrades, stream aborts, and fast-fail behavior.

If those items land, WUSP moves from “promising controlled transport” to “serious production protocol.” Without them, it remains a strong internal design that still asks too much good luck from the network.

13. Conclusion

WUSP gets the main idea exactly right. Do not build another public management plane around a tunnel that already authenticates, encrypts, and roams. Put management inside the tunnel. Keep the wire format small. Keep the data typed. Keep the controller practical.

That is what this implementation does. It is elegant in the places that matter most: placement, framing, typing, and operator workflow. It is less complete in the places that real networks eventually punish: adaptive sizing, deterministic fragmented-message recovery, and a finished transfer subsystem.

The result is a protocol that is already meaningful, already useful, and not yet fully done.

References

1. Broadband Forum, *TR-369 User Services Platform*, Issue 1. <https://www.broadband-forum.org/pdfs/tr-369-1-0-0.pdf>
2. WireGuard, *Protocol and Cryptography*. <https://www.wireguard.com/protocol/>
3. Jason A. Donenfeld, *WireGuard: Next Generation Kernel Network Tunnel*. <https://www.wireguard.com/papers/wireguard.pdf>
4. Eggert, Fairhurst, Shepherd, *RFC 8085: UDP Usage Guidelines*. <https://datatracker.ietf.org/doc/html/rfc8085>
5. Fairhurst, Jones, Tüxen, Rüngeler, Völker, *RFC 8899: Packetization Layer Path MTU Discovery for Datagram Transports*. <https://datatracker.ietf.org/doc/html/rfc8899>
6. Broadband Forum USP Data Models, *Device:2.20 TR-181 with WireGuard*. <https://usp-data-models.broadband-forum.org/tr-181-2-20-1-usp-diffs.html>
7. WireGuard, *Formal Verification*. <https://www.wireguard.com/formal-verification/>